

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук

должность, уч. степень, звание

подпись, дата

Н.В. Богословская

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

ПЛАТФОРМА LOGINOM

по курсу: БОЛЬШИЕ ДАННЫЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321

подпись, дата

И.А. Лаврешин

инициалы, фамилия

Санкт-Петербург 2026

1. Цель работы

Освоить построение и отбор признаков с использованием программного (Python) и визуального подходов.

2. Задание

Реализуйте рассмотренные в лабораторной работе этапы построения и отбора признаков для наборов данных как с использованием Python в соответствующем узле платформы, так и с помощью визуальных инструментов этой платформы. Сформулируйте аргументированные заключения о преимуществах или недостатках различных методов анализа.

3. Ход работы

В среде Loginom разработали сценарий, который решает поставленную задачу двумя разными методами: через интеграцию с Python и с использованием встроенных визуальных компонентов платформы. Результат представлен на рисунке 1.

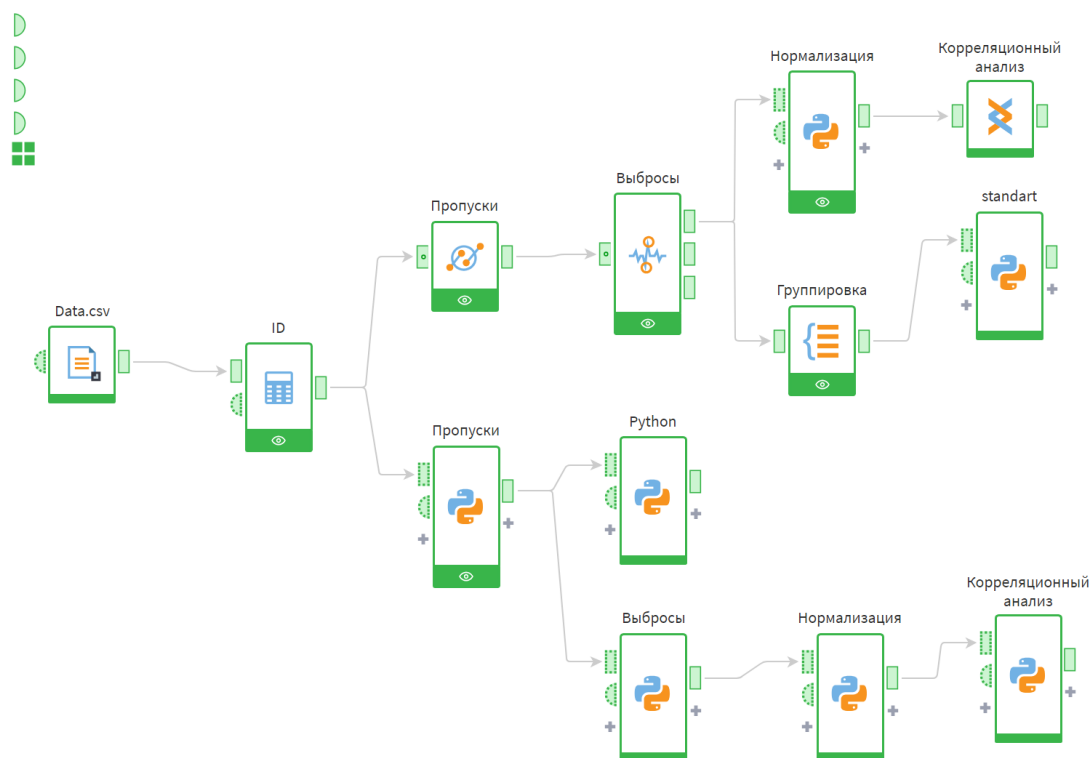


Рисунок 1 – Разработанный сценарий

Для исследования взяли набор данных по социально-экономическим и медицинским показателям разных стран за несколько лет. Среди ключевых параметров: продолжительность жизни, потребление алкоголя, смертность взрослых, образование, расходы на медицину, заболеваемость и другие факторы, влияющие на здоровье. После загрузки данных проверили наличие пропусков, что продемонстрировано на рисунках 2 – 3. Пробелы заполняли двумя способами: через скрипты Python и визуальные инструменты Loginom.

```

1 import builtin_data
2 from builtin_data import InputTable, OutputTable
3 import pandas as pd
4 from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
5
6 data = to_data_frame(InputTable)
7 data.columns = data.columns.str.strip()
8 #убранные столбцы
9 exclude_cols = ['Country', 'Status']
10 #обрабатываемые столбцы
11 cols = [col for col in data.columns if col not in exclude_cols]
12
13 for col in cols:
14     # привести к строке и убрать пробелы
15     data[col] = data[col].astype(str).str.strip()
16     #замена пустых значений и NaN
17     data[col] = data[col].replace(['', ' ', 'nan', 'NaN', 'None', 'NULL', 'null', '?'], pd.NA)
18     #перевести в число
19     data[col] = pd.to_numeric(data[col], errors='coerce')
20     #заполнение пропусков медианой
21     med = data[col].median()
22     data[col] = data[col].fillna(med)
23 output_frame = pd.DataFrame(data)
24 prepare_compatible_table(OutputTable, output_frame, with_index=False)
25 fill_table(OutputTable, output_frame, with_index=False)

```

Рисунок 2 – Заполнение пропусков через Python

№	Входные поля Фильтрация	Вид данных		Метод обработки
1	ID	Непрерывный	☐	Не выбран
2	Schooling	Непрерывный	☒	Заменять медианой
3	Life expectancy	Непрерывный	☐	Не выбран
4	Alcohol	Непрерывный	☒	Заменять медианой
5	BMI	Непрерывный	☒	Заменять медианой
6	GDP	Непрерывный	☒	Заменять медианой
7	Population	Непрерывный	☒	Заменять медианой
8	Country	Дискретный	☒	Заменять наиболее вероятным
9	Year	Непрерывный	☐	Не выбран
10	infant deaths	Непрерывный	☒	Заменять медианой
11	percentage expenditure	Непрерывный	☒	Заменять медианой
12	Measles	Непрерывный	☒	Заменять медианой
13	under-five deaths	Непрерывный	☒	Заменять медианой
14	HIV/AIDS	Непрерывный	☒	Заменять медианой
15	Hepatitis B	Непрерывный	☒	Заменять медианой
16	Adult Mortality	Непрерывный	☒	Заменять медианой
17	Polio	Непрерывный	☒	Заменять медианой
18	Total expenditure	Непрерывный	☒	Заменять медианой
19	Diphtheria	Непрерывный	☒	Заменять медианой
20	thinness 1-19 years	Непрерывный	☒	Заменять медианой
21	thinness 5-9 years	Непрерывный	☒	Заменять медианой
22	Income composition of resources	Непрерывный	☒	Заменять медианой

Рисунок 3 – Заполнение пропусков через визуальный инструмент

Удалили выбросы, мешающие анализу, что представлено на рисунках 4 –

5. Использовали Python – убрали значения за пределами стандартного отклонения, и Loginom – автоматически находил аномалии и заменял их медианой.

```

1 import builtin_data
2 from builtin_data import InputTable, OutputTable
3 import pandas as pd
4 from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
5 import numpy as np
6 data = to_data_frame(InputTable)
7 data.columns = data.columns.str.strip()
8
9 numeric_cols = data.select_dtypes(include=[np.number]).columns
10 x=3
11 for column in numeric_cols:
12     upper_lim = data[column]
13     mean = data[column].mean()
14     std = data[column].std()
15     upper_lim = mean+std*x
16     lower_lim = mean-std*x
17     data = data[(data[column] < upper_lim)& (data[column]>lower_lim)]
18 output_frame = data
19 prepare_compatible_table(OutputTable, output_frame, with_index=False)
20 fill_table(OutputTable, output_frame, with_index=False)

```

Рисунок 4 – Удаление выбросов через Python

Входные поля		Вид данных
Фильтрация		
☐	9.0 ID	Непрерывный
☐	ab Country	Дискретный
☐	12 Year	Непрерывный
☒	12 infant deaths	Непрерывный
☒	9.0 percentage expenditure	Непрерывный
☒	12 Measles	Непрерывный
☒	12 under-five deaths	Непрерывный
☒	9.0 HIV/AIDS	Непрерывный
☒	9.0 Schooling	Непрерывный
☒	9.0 Life expectancy	Непрерывный
☒	9.0 Alcohol	Непрерывный
☒	9.0 BMI	Непрерывный
☒	9.0 GDP	Непрерывный
☒	12 Population	Непрерывный
☒	12 Hepatitis B	Непрерывный
☒	12 Adult Mortality	Непрерывный
☒	12 Polio	Непрерывный
☒	9.0 Total expenditure	Непрерывный
☒	12 Diphtheria	Непрерывный
☒	9.0 thinness 1-19 years	Непрерывный
☒	9.0 thinness 5-9 years	Непрерывный
☒	9.0 Income composition of resources	Непрерывный

Рисунок 5 – Определение выбросов через визуальный инструмент

Следом за устранением аномалий провели нормализацию данных — уже в Python. Это нужно, чтобы привести разные показатели к единому масштабу и их можно было корректно сравнивать. Использовали метод масштабирования, переводящий все значения в интервал от 0 до 1. Соответствующий код показан на рисунке 6.

```

1 import builtin_data
2 from builtin_data import InputTable, OutputTable
3 import pandas as pd
4 from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
5 import numpy as np
6 data = to_data_frame(InputTable)
7
8 drop_cols = ['ID', 'Year']
9 numeric_cols = data.select_dtypes(include=[np.number]).columns
10 numeric_cols = [col for col in numeric_cols if col not in drop_cols ]
11 for column in numeric_cols:
12     col_min = data[column].min()
13     col_max = data[column].max()
14
15     if col_max != col_min:
16         data[column] = (data[column] - col_min) / (col_max - col_min)
17     else:
18         data[column] = 0
19 output_frame = pd.DataFrame(data)
20
21 prepare_compatible_table(OutputTable, output_frame, with_index=False)
22 fill_table(OutputTable, output_frame, with_index=False)

```

Рисунок 6 – Нормализация на Python

После приведения данных к единому масштабу построили график, на котором отразили нормализованные величины двух показателей – "Продолжительность жизни" и "Алкоголь".



Рисунок 7 – График нормализованных значений признаков

Затем выполнили стандартизацию данных средствами Python. Она нужна, чтобы все признаки оказались в едином масштабе: после обработки среднее каждого показателя становится равным нулю, а среднеквадратичное отклонение – единице. Код этого этапа представлен на рисунке 8.

```

1 import builtin_data
2 from builtin_data import InputTable, OutputTable
3 import pandas as pd
4 from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
5 import numpy as np
6 data = to_data_frame(InputTable)
7
8 drop_cols = ['ID', 'Year']
9 numeric_cols = data.select_dtypes(include=[np.number]).columns
10 numeric_cols=[col for col in numeric_cols if col not in drop_cols ]
11
12 for column in numeric_cols:
13     col_mean = data[column].mean()
14     col_std = data[column].std()
15     data[column] = (data[column] - col_mean) / col_std
16 output_frame = pd.DataFrame(data)
17
18 prepare_compatible_table(OutputTable, output_frame, with_index=False)
19 fill_table(OutputTable, output_frame, with_index=False)

```

Рисунок 8 – Стандартизация на Python

После стандартизации построили график для двух показателей — "Продолжительность жизни" и "Алкоголь". Он наглядно показывает, как изменились данные после приведения к единому масштабу. Диаграмма представлена на рисунке 9.

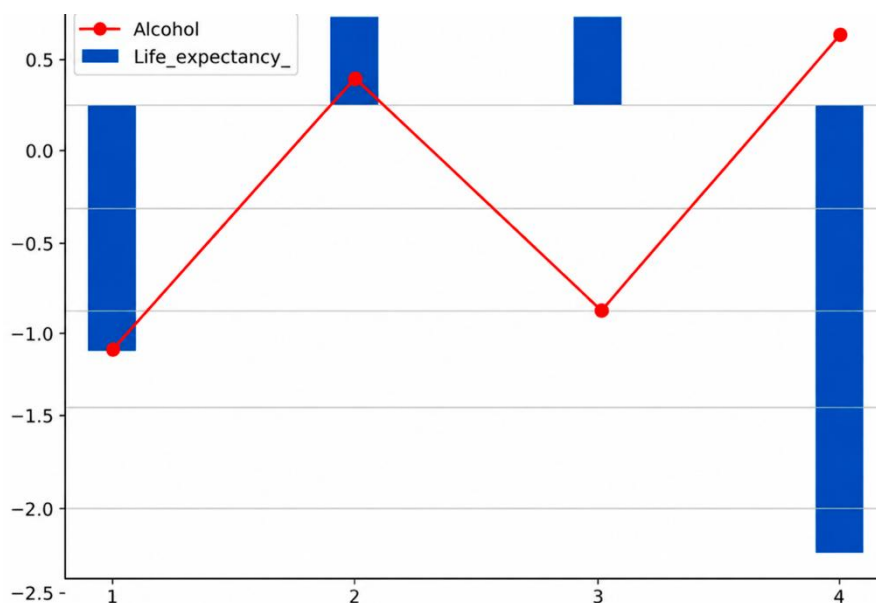


Рисунок 9 – График стандартизированных значений признаков

На финальном этапе провели корреляционный анализ — чтобы понять, как переменные связаны между собой. Степень взаимосвязи оценивали с помощью коэффициента Пирсона. Настройки в Logiном и код на Python показаны на рисунках 10 – 11.

Входные колонки	☒ Набор 1	☒ Набор 2
9.0 Measles_	☒	☐
9.0 _HIV_AIDS	☒	☐
9.0 Life_expectancy_	☐	☒
9.0 Alcohol	☒	☐
9.0 _BMI_	☒	☐
9.0 GDP	☒	☐
9.0 Hepatitis_B	☒	☐
9.0 ID	☐	☐
ab Country	☐	☐
12 Year	☐	☐
9.0 infant_deaths	☐	☐
9.0 percentage_expenditure	☐	☐
9.0 under_five_deaths_	☐	☐
9.0 Schooling	☐	☐
9.0 Population	☐	☐
9.0 COL41	☐	☐
9.0 COL42	☐	☐
9.0 COL43	☐	☐
9.0 COL44	☐	☐
9.0 COL45	☐	☐
9.0 COL46	☐	☐
9.0 COL47	☐	☐

Рисунок 10 – Настройки корреляции

```

1 import builtin_data
2 from builtin_data import InputTable, OutputTable
3 import pandas as pd
4 from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
5 import numpy as np
6 data = to_data_frame(InputTable)
7
8 numeric_cols = data.select_dtypes(include=[np.number]).columns
9
10 for col in data.columns:
11     if not pd.api.types.is_numeric_dtype(data[col]):
12         data[col] = pd.factorize(data[col])[0]
13
14 def correl(X_train, limit):
15     cor = X_train.corr(method='pearson')
16     print(cor)
17     return cor[abs(cor) > limit]
18
19 output_frame = correl(data, 0.0).reset_index()
20 output_frame.rename(columns={'index': 'Параметр'}, inplace=True)
21
22 prepare_compatible_table(OutputTable, output_frame, with_index=False)
23 fill_table(OutputTable, output_frame, with_index=False)

```

Рисунок 11 – Настройки корреляции через Python

#	ab Поле1.Имя	ab Поле1.Метка	ab Поле2.Имя	ab Поле2.Метка	9.0 Пирсона
1	Measles_	Measles_	Life_expectancy_	Life_expectancy_	-0,18
2	_HIV_AIDS	_HIV_AIDS	Life_expectancy_	Life_expectancy_	-0,57
3	Alcohol	Alcohol	Life_expectancy_	Life_expectancy_	0,39
4	_BMI_	_BMI_	Life_expectancy_	Life_expectancy_	0,56
5	GDP	GDP	Life_expectancy_	Life_expectancy_	0,40
6	Hepatitis_B	Hepatitis_B	Life_expectancy_	Life_expectancy_	0,24

Рисунок 12 – Результаты анализа корреляции